

ChatGPT 5.1 ✓

This is a copy of a conversation between ChatGPT & Anonymous.

Report conversation

Give the intuition behind [MHA / MQA / GQA / cross-attention / causal attention], following the intuition style used in the course discussion notes. Do NOT define formulas yet. Only focus on the high-level idea and what problem the mechanism is solving.

gave a clean high-level explanation of MHA/MQA/GQA/cross/causal following the “intuition only” constraint.

Below is the **intuition-only** explanation for **MHA / MQA / GQA / cross-attention / causal attention**, written in the **same high-level tone and structure as the course’s discussion notes**: focusing on *what problem each mechanism is solving, why it exists, what idea motivates it, without introducing notation or formulas yet*.

◆ Intuition Behind the Attention Mechanisms

(No formulas yet — only high-level reasoning)

1. Multi-Head Attention (MHA) — “Look from multiple perspectives”

What problem it tries to solve:

A single attention pattern tends to focus too narrowly — it can only capture **one kind of relationship** at a time. Natural language or images, however, contain **multiple kinds of structure simultaneously** (e.g., syntactic, semantic, positional, long-range, local).

Core idea / intuition:

Instead of one attention lens, use *multiple small lenses*, each learning to capture a **different relational pattern**.

Each head becomes a “specialist”:

- one head may focus on local context
- another detects subject–verb relations
- another tracks long-range dependencies

MHA = **parallel perspectives**, merged to form a richer representation.

2. Multi-Query Attention (MQA) — *“Speed up attention by sharing keys & values”*

What problem it tries to solve:

MHA is expensive because **every head keeps its own K and V**, so memory and compute scale linearly with the number of heads.

This becomes the bottleneck for long sequences.

Core idea / intuition:

MQA asks: *Do we really need different keys and values for each head?*

Empirically, the model mostly needs **multiple query projections** (multiple ways to read information), but the information being read (K/V) can be **shared**.

Thus:

- Many heads → many queries (diverse perspectives)
- But **only one set of keys and one set of values** → huge memory savings

MQA = “shared library, multiple readers.”

3. Grouped-Query Attention (GQA) — *“Middle ground between MHA and MQA”*

What problem it tries to solve:

MQA is efficient but can hurt model quality because sharing keys/values across too many heads reduces representational richness.

Core idea / intuition:

GQA is a **compromise**: group the heads into a few clusters.

- Each group shares K/V
- But different groups have different K/V sets

This keeps memory *much lower* than MHA but keeps representational diversity *much higher* than MQA.

GQA = “shared K/V, but not too shared.”

4. Cross-Attention — “Let one sequence read another sequence”

What problem it tries to solve:

Sometimes the model must **condition on external information**, not just its own past tokens. For example:

- In a Transformer encoder–decoder, the decoder needs to **consult the encoder output**.
- In retrieval-augmented models, the decoder must read **retrieved documents**.

Core idea / intuition:

Cross-attention allows one sequence (the “reader”) to **query** another sequence (the “source of information”).

It is essentially a *lookup mechanism*:

- “What part of the encoder output is relevant to the current decoding step?”
- “Which retrieved fact is helpful for answering this question?”

Cross-attention = “**attention over *another* sequence.**”

5. Causal (Masked) Attention — *“Don’t look into the future”*

What problem it tries to solve:

During autoregressive generation, the model must not use future information when predicting the next token — otherwise training becomes inconsistent with inference (“cheating”).

Core idea / intuition:

Causal masking enforces a **strict left-to-right information flow**:

- Token t can attend only to tokens $\leq t$
- No head is allowed to peek into future positions

This makes training match generation: the model only uses information it actually will have at inference time.

Causal attention = **“attention that respects time.”**

◆ Summary of Each Mechanism’s Purpose

Mechanism	Intuition	Problem Solved
MHA	Multiple perspectives from multiple small attention heads	Richer relational structure modeling
MQA	Share K/V across all heads	Reduce memory + improve inference speed
GQA	Partial sharing (between MHA and MQA)	Keep quality near MHA but efficiency near MQA
Cross-Attention	One sequence attending to another	Conditioning on external information (encoder → decoder, retrieval, etc.)
Causal Attention	No future information allowed	Enables autoregressive generation

If you want, I can now proceed (course-style) with:

1. **Notation definition** using lecture-note conventions,
2. **Step-by-step formula derivation**,
3. **Complexity comparison** ($O(H \cdot N^2)$, $O(N^2)$, etc.),
4. **Comparison across mechanisms (MHA vs MQA vs GQA)**,
5. **Kernelized/linear attention intuition and complexity reduction.**

Would you like the next section in that order?

Now set up the notation **exactly**

define Q, K, V

define dimensions d, d_k, d_v

define number of heads h (if applicable)

I recognize that I made a mistake by specifying the notation explicitly in the notation-setup section, instead of allowing GPT to extract the notation from the attached homework-solution screenshots.

Now set up the notation **exactly**

define Q, K, V

define dimensions d, d_k, d_v

define number of heads h (if applicable)

define merge/reshape conventions

define assumptions such as sequence length n

Be consistent with the course notation; do NOT invent new symbols.

Below is the **notation setup exactly in the style used in your course lecture notes +**

discussion notes + homework.

I will **not** introduce formulas yet — only define the symbols and structural conventions the course uses.

Notation Setup (Exactly matching course conventions)

Your course uses **one consistent family of symbols** for attention:

- **Q, K, V** always denote the **query / key / value matrices**
- Sequence length is **n**
- Model (embedding) dimension is **d**
- Head dimension is **d_k** for keys/queries and **d_v** for values
- Number of heads is **h**
- Multi-head concatenation produces dimension **h·d_v** before projection
- Linear projections use capital **W** with subscripts
- Reshaping follows the **(n × d) → (h × n × d_k)** convention used in discussion notes

correctly set up Q, K, V, d, d_k, d_v, head h, reshape conventions, KV-group notation for GQA, and encoder/decoder notation.

Below is the exact structure.

1. Sequence and Embedding Dimensions

- Sequence length:

$$n$$

- Model (embedding) dimension:

$$d$$

Input representation (one layer's input):

$$X \in \mathbb{R}^{n \times d}$$

This matches both lecture and discussion notes.

2. Attention Projections

For any attention mechanism, the model computes learned linear projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Where:

- $W_Q \in \mathbb{R}^{d \times d_k h}$
- $W_K \in \mathbb{R}^{d \times d_k h}$
- $W_V \in \mathbb{R}^{d \times d_v h}$

The expanded dimensions depend on whether it is MHA, MQA, or GQA, but the course always begins with the full MHA shape and then specifies sharing.

3. Head Dimensions

Your course uses:

- Query/Key head dimension:

$$d_k$$

- Value head dimension:

$$d_v$$

For multi-head attention, each head has:

$$Q^{(i)} \in \mathbb{R}^{n \times d_k}, \quad K^{(i)} \in \mathbb{R}^{n \times d_k}, \quad V^{(i)} \in \mathbb{R}^{n \times d_v}$$

4. Number of Heads

$$h = \text{number of attention heads}$$

The projections produce tensors that are then **reshaped into h heads**:

$Q \rightarrow \text{reshape to } (h, n, d_k)$

$K \rightarrow \text{reshape to } (h, n, d_k)$

$V \rightarrow \text{reshape to } (h, n, d_v)$

This reshape convention is explicitly used in the discussion notes:

- First dimension = head index
- Second = sequence length
- Third = per-head dimension

5. Merge / Concatenation Convention

After computing attention for each head, the outputs are concatenated along the feature dimension:

$$\text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{n \times (hd_v)}$$

Then a linear output projection is applied:

$$W_O \in \mathbb{R}^{hd_v \times d}$$

This returns the representation to dimension d .

6. Specialized Mechanisms (Notation Only)

Multi-Head Attention (MHA)

Each head has its own $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}$.

Equivalently implemented via block structure within W_Q, W_K, W_V .

Multi-Query Attention (MQA)

Only the **queries** have h separate projections:

- Q: h different heads
- K, V: 1 shared projection each

Thus:

$$K, V \in \mathbb{R}^{n \times d_k}, \text{ (1 head)}$$

$$Q \in \mathbb{R}^{h \times n \times d_k}$$

Grouped-Query Attention (GQA)

Keys/values are shared **within groups**:

Let:

$$g = \text{number of KV groups}, \quad g < h$$

Then:

- Q has h heads
- K, V have g sets of projections

Shapes use:

$$K \in \mathbb{R}^{g \times n \times d_k}, \quad V \in \mathbb{R}^{g \times n \times d_v}$$

7. Cross-Attention Notation

Two sequences:

- Decoder-side (queries):

$$X_{dec} \in \mathbb{R}^{n_{dec} \times d}$$

- Encoder outputs (keys/values):

$$X_{enc} \in \mathbb{R}^{n_{enc} \times d}$$

Then:

$$Q = X_{dec}W_Q, \quad K = X_{enc}W_K, \quad V = X_{enc}W_V$$

Reshaping follows the same conventions as MHA.

8. Causal (Masked) Attention Notation

Sequence length is again:

$$n$$

Mask matrix:

$$M \in \{0, -\infty\}^{n \times n}$$

Where:

- $M_{i,j} = 0$ if $j \leq i$
- $M_{i,j} = -\infty$ if $j > i$

This mask is later added to attention logits before softmax.

Everything above matches course notation exactly:

- Matrix shapes match discussion worksheets
- h, d, d_k, d_v, n are consistent
- Reshaping convention ($h \times n \times d_k$) follows the notes
- Cross-attention uses encoder/decoder X_{enc}, X_{dec}
- Causal masks use the upper-triangular $-\infty$ mask

If you'd like, the next step (as required by the course structure) is:

✓ **Step-by-step formula derivation for**

MHA, MQA, GQA, cross-attention, causal attention

> Now derive the formulas step-by-step, following the style of the course:

>

- start from Q, K, V
- express attention scores
- show softmax
- expand multi-head (or multi-query / group-query) structure
- show how projections differ
- show output combination

provided explicit step-by-step formulas beginning from $Q, K, V \rightarrow \text{scores} \rightarrow \text{softmax} \rightarrow \text{per-head expansion} \rightarrow \text{multi-head merge}$

Thought for 21s >

Let's go through the **formula derivations step-by-step**, in the same order and style your course uses:

1. start from Q, K, V
2. form attention scores
3. apply softmax
4. expand to multiple heads / queries / groups
5. show how projections differ
6. combine outputs

I'll keep the notation consistent with what we already fixed:

- sequence length n (or $n_{\text{enc}}, n_{\text{dec}}$)

- model dim d , head dims d_k, d_v
 - number of heads h (and possibly KV groups g)
-

1. Base: Single-Head Scaled Dot-Product Attention

1.1 Start from Q, K, V

Given input $X \in \mathbb{R}^{n \times d}$,

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

with

$$W_Q \in \mathbb{R}^{d \times d_k}, \quad W_K \in \mathbb{R}^{d \times d_k}, \quad W_V \in \mathbb{R}^{d \times d_v}$$

so

$$Q, K \in \mathbb{R}^{n \times d_k}, \quad V \in \mathbb{R}^{n \times d_v}.$$

1.2 Attention scores (logits)

For each pair of positions (i, j) , we compute a similarity score between query at i and key at j :

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{n \times n}.$$

- $S_{ij} = \frac{\langle Q_i, K_j \rangle}{\sqrt{d_k}}.$

1.3 Softmax → attention weights

Apply softmax **row-wise** (over the key dimension j):

$$A_{ij} = \frac{\exp(S_{ij})}{\sum_{j'=1}^n \exp(S_{ij'})}, \quad A \in \mathbb{R}^{n \times n}.$$

So each row $A_{i,:}$ is a probability distribution over positions j that the i -th query attends to.

1.4 Weighted sum of values

The output of single-head attention is:

$$\text{Attn}(Q, K, V) = AV \in \mathbb{R}^{n \times d_v}.$$

Each output vector at position i is

$$(\text{Attn}(Q, K, V))_i = \sum_{j=1}^n A_{ij} V_j.$$

2. Multi-Head Attention (MHA)

Now we replicate this **per head** and then combine.

2.1 Per-head projections

Given the same input $X \in \mathbb{R}^{n \times d}$, for head $i \in \{1, \dots, h\}$:

$$Q^{(i)} = XW_Q^{(i)}, \quad K^{(i)} = XW_K^{(i)}, \quad V^{(i)} = XW_V^{(i)},$$

with

$$W_Q^{(i)}, W_K^{(i)} \in \mathbb{R}^{d \times d_k}, \quad W_V^{(i)} \in \mathbb{R}^{d \times d_v}.$$

You can also think of a single big W_Q, W_K, W_V which are then reshaped into head blocks, but per-head matrices is conceptually simpler.

So shape-wise:

$$Q^{(i)}, K^{(i)} \in \mathbb{R}^{n \times d_k}, \quad V^{(i)} \in \mathbb{R}^{n \times d_v}.$$

2.2 Per-head attention

For each head i ,

$$S^{(i)} = \frac{Q^{(i)}(K^{(i)})^\top}{\sqrt{d_k}} \in \mathbb{R}^{n \times n},$$

$$A_{ij}^{(i)} = \frac{\exp(S_{ij}^{(i)})}{\sum_{j'} \exp(S_{ij'}^{(i)})}, \quad A^{(i)} \in \mathbb{R}^{n \times n},$$

$$\text{head}_i = A^{(i)} V^{(i)} \in \mathbb{R}^{n \times d_v}.$$

2.3 Concatenate and project back

Concatenate all heads along the feature dimension:

$$H = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{n \times (hd_v)}.$$

Then apply an output projection:

$$\text{MHA}(X) = HW_O, \quad W_O \in \mathbb{R}^{hd_v \times d}, \quad \text{MHA}(X) \in \mathbb{R}^{n \times d}.$$

So the full MHA layer maps $\mathbb{R}^{n \times d} \rightarrow \mathbb{R}^{n \times d}$.

3. Multi-Query Attention (MQA)

In MQA, we keep **many query heads**, but **share** keys and values across all of them.

3.1 Projections

Given $X \in \mathbb{R}^{n \times d}$:

- Queries still have h heads:

$$Q^{(i)} = XW_Q^{(i)} \in \mathbb{R}^{n \times d_k}, \quad i = 1, \dots, h.$$

- Keys and values are **shared**, i.e. only **one** K and V:

$$K = XW_K, \quad V = XW_V,$$

with

$$W_K \in \mathbb{R}^{d \times d_k}, \quad W_V \in \mathbb{R}^{d \times d_v},$$

so

$$K \in \mathbb{R}^{n \times d_k}, \quad V \in \mathbb{R}^{n \times d_v}.$$

3.2 Attention per query head (shared K,V)

For each head i :

$$S^{(i)} = \frac{Q^{(i)}K^\top}{\sqrt{d_k}} \in \mathbb{R}^{n \times n},$$

$$A_{ij}^{(i)} = \frac{\exp(S_{ij}^{(i)})}{\sum_{j'} \exp(S_{ij'}^{(i)})},$$

$$\text{head}_i = A^{(i)}V \in \mathbb{R}^{n \times d_v}.$$

Only difference from MHA: K, V no longer depend on i .

3.3 Output combination

Same as MHA:

$$H = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{n \times (hd_v)},$$

$$\text{MQA}(X) = HW_O, \quad W_O \in \mathbb{R}^{hd_v \times d}.$$

So structurally it **looks identical at the output**, but inside, K and V are shared across heads.

4. Grouped-Query Attention (GQA)

GQA interpolates between MHA and MQA:

- h query heads
- only g **groups** of K,V projections, where $g < h$
- each group's K,V are shared among some subset of heads.

Let:

- g = number of KV groups
- assume each group has $\frac{h}{g}$ query heads (for simplicity).

4.1 Projections

Given $X \in \mathbb{R}^{n \times d}$:

- For each **query head** $i = 1, \dots, h$:

$$Q^{(i)} = XW_Q^{(i)} \in \mathbb{R}^{n \times d_k}.$$

- For each **KV group** $r = 1, \dots, g$:

$$K^{(r)} = XW_K^{(r)}, \quad V^{(r)} = XW_V^{(r)},$$

with

$$W_K^{(r)} \in \mathbb{R}^{d \times d_k}, \quad W_V^{(r)} \in \mathbb{R}^{d \times d_v},$$

so

$$K^{(r)} \in \mathbb{R}^{n \times d_k}, \quad V^{(r)} \in \mathbb{R}^{n \times d_v}.$$

Each head i belongs to some group $r(i) \in \{1, \dots, g\}$, and shares K,V with the other heads in that group.

4.2 Attention per head with grouped K,V

For each head i , using its group's K,V:

$$S^{(i)} = \frac{Q^{(i)}(K^{(r(i))})^\top}{\sqrt{d_k}},$$

$$A_{ij}^{(i)} = \frac{\exp(S_{ij}^{(i)})}{\sum_{j'} \exp(S_{ij'}^{(i)})},$$

$$\text{head}_i = A^{(i)} V^{(r(i))}.$$

If $g = h$, we recover MHA.

If $g = 1$, we recover MQA.

4.3 Output combination

Again, same concatenation and projection:

$$H = \text{Concat}(\text{head}_1, \dots, \text{head}_h),$$

$$\text{GQA}(X) = HW_O, \quad W_O \in \mathbb{R}^{hd_v \times d}.$$

5. Cross-Attention (Encoder–Decoder)

Only difference: **queries** come from one sequence, **keys/values** from another.

Let:

- Encoder outputs: $X_{\text{enc}} \in \mathbb{R}^{n_{\text{enc}} \times d}$
- Decoder hidden states: $X_{\text{dec}} \in \mathbb{R}^{n_{\text{dec}} \times d}$.

5.1 Projections (per head, as in MHA)

For head i :

$$Q^{(i)} = X_{\text{dec}} W_Q^{(i)} \in \mathbb{R}^{n_{\text{dec}} \times d_k},$$

$$K^{(i)} = X_{\text{enc}} W_K^{(i)} \in \mathbb{R}^{n_{\text{enc}} \times d_k},$$

$$V^{(i)} = X_{\text{enc}} W_V^{(i)} \in \mathbb{R}^{n_{\text{enc}} \times d_v}.$$

5.2 Attention scores and outputs

Now scores are between decoder positions and encoder positions:

$$S^{(i)} = \frac{Q^{(i)} (K^{(i)})^\top}{\sqrt{d_k}} \in \mathbb{R}^{n_{\text{dec}} \times n_{\text{enc}}},$$

$$A_{t,j}^{(i)} = \frac{\exp(S_{t,j}^{(i)})}{\sum_{j'} \exp(S_{t,j'}^{(i)})}, \quad t \in \{1, \dots, n_{\text{dec}}\}, j \in \{1, \dots, n_{\text{enc}}\},$$

$$\text{head}_i = A^{(i)} V^{(i)} \in \mathbb{R}^{n_{\text{dec}} \times d_v}.$$

Concatenate over heads and project back as before:

$$H = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{n_{\text{dec}} \times (hd_v)},$$

$$\text{CrossAttn}(X_{\text{dec}}, X_{\text{enc}}) = HW_O.$$

6. Causal (Masked) Attention

Causal attention modifies the **scores** before softmax using a **mask**.

6.1 Causal mask

For a sequence of length n , define mask $M \in \mathbb{R}^{n \times n}$:

$$M_{ij} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

(Conceptually, “disallow” attention to the future.)

6.2 Apply mask to scores

Given standard scores $S = \frac{QK^\top}{\sqrt{d_k}}$ (single head or per head), define:

$$\tilde{S} = S + M.$$

Then

$$A_{ij} = \frac{\exp(\tilde{S}_{ij})}{\sum_{j'} \exp(\tilde{S}_{ij'})}.$$

Because $\exp(-\infty) = 0$, positions $j > i$ get zero probability.

Per head it’s the same idea:

$$\tilde{S}^{(i)} = S^{(i)} + M, \quad A^{(i)} = \text{softmax}(\tilde{S}^{(i)} \text{ row-wise}).$$

Then the rest (multiplying by $V^{(i)}$, concatenation, projection) is unchanged.

If you’d like, next we can:

- write down the **time / memory complexity** for
 - vanilla MHA
 - MQA
 - GQA
 - cross-attention
 - causal attention

in the exact asymptotic style used in your homework (e.g. $O(hn^2d_k)$, etc.),

- and then **compare them** systematically, like in the “computational complexity of attention” question set from your course.

>please give the computational complexity of this
attention mechanism using the
> *same assumptions*
>
- sequence length n
- hidden size d
- head dimension d_k

Below is the **computational complexity analysis** written in the **exact style used in your course's discussion + homework**:

- use the same assumptions:
 - sequence length n
 - hidden size d

- per-head dimension d_k (and $d_v \sim d_k$)
- number of heads h
- use **the same accounting conventions** as the course:
 counting matrix multiplies with shapes $(n \times d)(d \times d_k)$,
 attention scores $(n \times d_k)(d_k \times n)$,
 softmax cost $O(n^2)$ per head, etc.

I keep the structure exactly parallel across mechanisms.

◆ 0. Preliminaries (same assumptions as lecture/discussion)

- Sequence length: n
 - Hidden size: d
 - **heads: h**
 - Head dimension: d_k (and d_v treated as d_k)
 - Assume $hd_k = d$ unless otherwise noted.
 - All complexity is **per layer**, ignoring constant factors but keeping h, n, d, d_k .
-

◆ 1. Multi-Head Attention (MHA)

1.1 Q, K, V projections

Each: XW , shape

$$(n \times d)(d \times hd_k) = O(ndhd_k) = O(nd^2).$$

All three projections together:

$$O(nd^2).$$

1.2 Attention score computation

For each head i :

$$Q^{(i)} K^{(i)\top} : (n \times d_k)(d_k \times n) = O(n^2 d_k).$$

Across all h heads:

$$O(hn^2 d_k) = O(n^2 d).$$

1.3 Softmax

Softmax over each row: cost $O(n)$ per row, total $O(n^2)$ per head.

Across all heads:

$$O(hn^2).$$

(Usually dominated by $n^2 d_k$.)

1.4 Weighted sum with V

Per head:

$$A^{(i)} V^{(i)} : (n \times n)(n \times d_k) = O(n^2 d_k).$$

Across all heads:

$$O(hn^2 d_k) = O(n^2 d).$$

1.5 Output projection

Concatenated matrix has size $n \times (hd_k)$. Multiplying by $W_O \in \mathbb{R}^{hd_k \times d}$:

$$(n \times hd_k)(hd_k \times d) = O(nd^2).$$

given the notation I supplied, GPT produced results that were equivalent to those in the homework solution.

Total cost for MHA

Dominant term is the n^2 attention computation:

$$O(nd^2) + O(n^2d)$$

Since for transformers typically $d \gg d_k$ but $hd_k = d$, the standard simplified form (same as course) is:

$$O(n^2d)$$

Solution:

- i. $O((Md + M^2)dh)$
- ii. $O((Md + M^2)dh)$
- iii. $O((Md + Nd + MN)hd)$
- iv. $O((Nd + N^2)dh)$

◆ 2. Multi-Query Attention (MQA)

MQA keeps h queries, but only 1 shared K and 1 shared V.

This changes the complexity of the n^2 terms.

2.1 Q projections

Same as MHA (still have h query heads):

$$O(nd^2).$$

2.2 K, V projections

One K and one V:

$$(n \times d)(d \times d_k) = O(ndd_k)$$

for each.

Together:

$$O(ndd_k) = O(nd^2/h).$$

This is smaller than MHA's $O(nd^2)$.

2.3 Attention scores

For each head:

$$(n \times d_k)(d_k \times n) = O(n^2 d_k).$$

Across all h heads, with **same K** reused:

$$h \cdot O(n^2 d_k) = O(n^2 d).$$

Same as MHA.

2.4 Softmax

Same as MHA:

$$O(hn^2).$$

2.5 Weighted sum with shared V

Same as scores:

$$O(hn^2 d_k) = O(n^2 d).$$

2.6 Output projection

Same as MHA:

$$O(nd^2).$$

Total cost for MQA

$$O(nd^2) + O(n^2d)$$

Same asymptotic flops as MHA,
but K/V memory and bandwidth reduced by factor h ,
which is why MQA improves speed in practice.

Solution:

- i. $O((Md + M^2)dh)$
- ii. $O((Md + M^2)dh)$
- iii. $O((Md + Nd + MN)hd)$
- iv. $O((Nd + N^2)dh)$

◆ 3. Grouped-Query Attention (GQA)

Let:

- h = total queries
- g = number of K/V groups
- Typically $g \ll h$.

Effect:

- Q projection: same as MHA
- K, V projection: cost scales with g instead of h
- Attention score computation still scales with h , like MHA

3.1 Q projections

Same:

$$O(nd^2).$$

3.2 K, V projections

One K, V per group $\rightarrow g$ groups:

$$g \cdot (n d d_k) = O(g n d_k d) = O\left(n d^2 \frac{g}{h}\right)$$

(using $h d_k = d$).

This is between MHA and MQA.

3.3 Attention scores

Heads still = h :

$$O(h n^2 d_k) = O(n^2 d).$$

3.4 Softmax

Same:

$$O(h n^2).$$

3.5 Weighted sums

Same:

$$O(h n^2 d_k) = O(n^2 d).$$

3.6 Output projection

Same:

$$O(n d^2).$$

Total cost for GQA

$$O(nd^2) + O(n^2d)$$

K/V projection term becomes:

$$O\left(nd^2 \cdot \frac{g}{h}\right)$$

which interpolates:

- **MHA** when $g = h$
- **MQA** when $g = 1$

◆ 4. Cross-Attention

Let decoder length = n_{dec} , encoder length = n_{enc} .

Using same per-head dimension d_k .

Key idea:

The cost becomes $O(n_{\text{dec}}n_{\text{enc}}d_k)$
instead of $O(n^2d_k)$.

4.1 Q, K, V projections

Decoder Q:

$$(n_{\text{dec}}d)(d \times hd_k) = O(n_{\text{dec}}d^2).$$

Encoder K,V:

$$(n_{\text{enc}}d)(d \times hd_k) = O(n_{\text{enc}}d^2).$$

4.2 Attention scores

Per head:

$$(n_{\text{dec}} \times d_k)(d_k \times n_{\text{enc}}) = O(n_{\text{dec}} n_{\text{enc}} d_k).$$

Across all heads:

$$O(n_{\text{dec}} n_{\text{enc}} d).$$

4.3 Softmax

Row-wise over encoder length:

$$O(n_{\text{dec}} n_{\text{enc}}).$$

(usually dominated)

4.4 Weighted sum

Per head:

$$(n_{\text{dec}} \times n_{\text{enc}})(n_{\text{enc}} \times d_k) = O(n_{\text{dec}} n_{\text{enc}} d_k).$$

Across h heads:

$$O(n_{\text{dec}} n_{\text{enc}} d).$$

4.5 Output projection

$$O(n_{\text{dec}} d^2).$$

Total cost for Cross-Attention

$$O(n_{\text{dec}}d^2 + n_{\text{enc}}d^2) + O(n_{\text{dec}}n_{\text{enc}}d)$$

Solution:

- i. $\mathcal{O}((Md + M^2)dh)$
- ii. $\mathcal{O}((Md + M^2)dh)$
- iii. $\mathcal{O}((Md + Nd + MN)hd)$
- iv. $\mathcal{O}((Nd + N^2)dh)$

◆ 5. Causal Attention

Causal = **same computation as MHA**, but masking does **not** change flop count.

Masking only zeroes out some logits; computing them still costs the same.

So complexity is identical to MHA:

$$O(nd^2) + O(n^2d)$$

✓ Final Summary Table

(Matches course's summary tables)

Mechanism	Complexity
MHA	$O(nd^2 + n^2d)$
MQA	$O(nd^2 + n^2d)$ (lower KV memory)
GQA	$O(nd^2 \frac{g}{h} + nd^2) + O(n^2d)$
Cross-Attention	$O(n_{\text{dec}}d^2 + n_{\text{enc}}d^2) + O(n_{\text{dec}}n_{\text{enc}}d)$
Causal Attention	$O(nd^2 + n^2d)$

All keep the same asymptotic computational cost in flops, with differences mainly in **memory, KV bandwidth, and constants**, exactly as your course notes emphasize.

If you want, I can now give the **final comparison reasoning** (the "why MQA/GQA

reduces cost at inference" part) in the same structure used in your course's "complexity-of-attention" worksheet.

> Compare this mechanism with the baseline (usually MHA) across:

>

- parameter count
- computational cost
- memory cost
- qualitative trade-offs

> Compare this mechanism with the baseline (usually MHA) across:

>

- parameter count
- computational cost
- memory cost
- qualitative trade-offs

Thought for 32s >

Nice, we've now reached the "compare with baseline MHA" step the course likes. I'll treat **MHA as the baseline** and compare:

- **MQA**
- **GQA**
- **Cross-attention**
- **Causal attention**

under the *same assumptions*:

- sequence length n
- hidden size d
- head dim d_k with $hd_k = d$

I'll go metric by metric.

1. Baseline: MHA (for reference)

- **Parameters (per layer)**
 - Q,K,V projections: each $d \times d$ (since $hd_k = d$)
 $\Rightarrow 3d^2$

- Output projection $W_O : d \times d$
 $\Rightarrow d^2$
 - **Total** (ignoring biases): $\approx 4d^2$
 - **Compute**
 - Complexity: $O(nd^2 + n^2d)$ per layer
 - n^2d term from attention scores + value mix
 - **Memory (during autoregressive decoding, KV cache)**
 - Store K,V for each of the h heads
 - Per token: $O(hd_k) = O(d)$
 - For sequence of length n : $O(nd)$ per layer
 - **Qualitative**
 - Maximum flexibility: each head has its own Q,K,V
 - Good quality, but **heaviest** in KV memory
-

2. MQA vs MHA

Key change: many query heads, **one shared K and shared V**.

2.1 Parameter count

- Q: still $d \times d$ (h query heads)
- K, V: now only **one** set each: $d \times d_k \approx d \times (d/h)$

So:

- Q: d^2
- K + V: $2dd_k = 2d^2/h$
- Output W_O : d^2

Total:

$$\text{Params(MQA)} \approx d^2 + 2\frac{d^2}{h} + d^2 = 2d^2 + \frac{2d^2}{h}$$

Compared to MHA's $\approx 4d^2$, MQA **saves parameters**, especially in K/V.

Summary: fewer parameters than MHA, with the gap larger for large h .

2.2 Computational cost

- Q projection: same $O(nd^2)$
- K,V projections: cheaper ($O(nd^2/h)$), but this is a **lower-order term**
- Attention scores and AV multiplication: still $O(n^2d)$ (one set of K,V reused across heads, but we still compute h score matrices)

Asymptotically: same big- O as MHA:

$$O(nd^2 + n^2d)$$

but **slightly less work in projections**.

2.3 Memory cost (KV cache)

- MHA KV cache: $O(nd)$ per layer
(K,V shape: $n \times h \times d_k$)
- MQA KV cache: K,V are **shared**, so shape:
 $n \times d_k = n \times (d/h)$

So:

$$\text{KV_mem(MQA)} = O\left(\frac{nd}{h}\right)$$

Memory savings of factor $\approx h$ in the K/V cache compared to MHA.

This is the *main* win of MQA, especially for large h and long context.

2.4 Qualitative trade-offs vs MHA

- **Pros**
 - Much smaller KV cache → faster autoregressive decoding, less memory
 - Important for long-context LLMs and many layers
 - **Cons**
 - Less flexibility: all heads see **the same K/V**
→ heads differ only through Q projections
 - Can slightly degrade model quality vs full MHA (especially at smaller scales)
 - **Use case intuition**
 - Good trade-off when **inference efficiency** is critical and some loss in flexibility is acceptable.
-

3. GQA vs MHA

Key change: h query heads but only g groups of K,V ($1 < g < h$).

3.1 Parameter count

- Q: same as MHA: d^2
- K,V: only g groups instead of h :

$$\text{Params}(K,V) = 2g(d \times d_k) \approx 2g \frac{d^2}{h}$$

- Output W_O : d^2

Total:

$$\text{Params}(\text{GQA}) \approx d^2 + 2g \frac{d^2}{h} + d^2 = 2d^2 + 2d^2 \frac{g}{h}.$$

Compare:

- $g = h \rightarrow \text{MHA}$
- $g = 1 \rightarrow \text{close to MQA}$

Summary: parameter count between MHA and MQA, controlled by g .

3.2 Computational cost

- Q projection: $O(nd^2)$ (same)
- K,V projection: $O(nd^2 \cdot g/h)$
- Attention scores + AV: still $O(n^2d)$ (we still have h heads)

Big- O : same as MHA:

$$O(nd^2 + n^2d)$$

but less work for K,V projections as g decreases.

3.3 Memory cost (KV cache)

- Each of the g K/V groups is cached, not each of the h heads.
- KV cache: $O(ngd_k) = O\left(nd \cdot \frac{g}{h}\right)$.

Interpolates between:

- "MHA: $g = h \rightarrow O(nd)$ "
- "MQA: $g = 1 \rightarrow O(nd/h)$ "

3.4 Qualitative trade-offs vs MHA

- **Pros**
 - Compromise between full flexibility and MQA efficiency
 - Often preserves quality closer to MHA while gaining much of MQA's KV savings
- **Cons**
 - Still less flexible than MHA (K,V shared within each group)
 - More complex implementation/hyperparameters (g to choose)
- **Use case intuition**
 - Good when you want **most of MHA's quality** but need **substantial KV memory savings**.

4. Cross-Attention vs Self MHA

Baseline here is **self-attention** MHA on a single sequence of length n .

4.1 Parameter count

- Self MHA: Q,K,V projections all from the *same* X .
- Cross-attention:
 - Q projection from decoder input X_{dec} : $d \times d$
 - K,V from encoder output X_{enc} : each $d \times d$
 - Output projection $d \times d$

In practice, cross-attention uses **separate** projection matrices from self-attention, but each is still $O(d^2)$.

So per *module*, cross-attention has **similar parameter count** to a self-attention block of the same width d .

4.2 Computational cost

If we compare self-attention on sequence length n vs cross-attention:

- | | |
|---|--|
| <ul style="list-style-type: none"> • Self MHA:
$O(nd^2 + n^2d)$ • Cross-attention:
n_{dec} queries and n_{enc} keys: | <p>Solution:</p> <ul style="list-style-type: none"> i. $O((Md + M^2)dh)$ ii. $O((Md + M^2)dh)$ iii. $O((Md + Nd + MN)hd)$ iv. $O((Nd + N^2)dh)$ |
|---|--|

$$O(n_{\text{dec}}d^2 + n_{\text{enc}}d^2 + n_{\text{dec}}n_{\text{enc}}d)$$

When $n_{\text{dec}} \approx n_{\text{enc}} \approx n$, same order as self-attention.

When n_{enc} is fixed and small, cross-attention can be cheaper.

4.3 Memory cost (KV cache)

For decoding:

- **Self MHA (decoder):** cache K,V for decoder-side self-attention.
- **Cross-attention:** cache K,V from encoder (fixed after encoding), then reuse them for all decoding steps.

Cross-attention's KV cache scales with $n_{\text{enc}}d$, independent of generated length.
So per decoding step, there's no extra growth beyond the encoder length.

4.4 Qualitative trade-offs vs self-attention

- **Pros**
 - Lets decoder condition on external sequence (encoder outputs, retrieved docs, etc.)
 - Naturally separates "source encoding" from "target generation"
- **Cons**
 - Slightly more parameters (because you have *both* self-attention and cross-attention modules in the decoder)
 - Extra complexity in architecture
- **Use case intuition**
 - Essential in **encoder-decoder** models (translation, seq2seq tasks).
 - Less crucial in **pure decoder-only LLMs** (they tend to use self-attention only).

5. Causal Attention vs Full (Bidirectional) MHA

Here, the **mechanism is the same** as MHA, but we apply a **causal mask**.

5.1 Parameter count

- Exactly the same as MHA: Q,K,V, W_O unchanged.
- Mask has no trainable parameters.

5.2 Computational cost

Naive implementation:

- Compute full score matrix $S \in \mathbb{R}^{n \times n}$
- Add mask M , apply softmax
- Complexity unchanged: $O(n^2d)$

It's reasonable to skip the computation, but showing how the triangular structure works would have improved clarity. I should have attached relevant materials.

(Some specialized implementations may exploit triangular structure, but in the standard course assumption, we **don't** count that.)

5.3 Memory cost

- Same KV cache size as non-causal MHA (per layer: $O(nd)$)
- Mask itself is negligible (often implicit, not stored as full $n \times n$ tensor).

5.4 Qualitative trade-offs vs full MHA

- **Pros**
 - Enforces left-to-right dependency → aligns training with autoregressive generation
 - Needed for language modeling / decoding
- **Cons**
 - Cannot use future tokens → weaker for tasks where full bidirectional context helps (e.g., masked LM)
- **Use case intuition**
 - Causal attention for **generators** (decoder-only LMs, autoregressive models)
 - Full (non-causal) attention for **encoders** (bidirectional understanding).

6. Big-Picture Summary (Course-Style)

Relative to baseline **MHA**:

Mechanism	Params vs MHA	Compute vs MHA	KV Memory vs MHA	Qualitative Trade-off
MQA	↓ K,V params	$\approx$ same $O(n^2d)$	↓ by $\sim h$	Efficiency-oriented, some loss of head diversity
GQA	Between MHA & MQA	$\approx$ same $O(n^2d)$	Between MHA & MQA	Balance between quality and KV efficiency
Cross-Attn	Similar per block	$O(n_{\text{dec}}n_{\text{enc}}d)$	KV fixed to encoder length	Enables conditioning on external sequence
Causal Attn	Same	Same big- O	Same	Enforces autoregressive constraint, no future peeking

The final comparative summary of the mechanisms is particularly helpful.

If you'd like, next we can add **kernelized / linear attention** to the same comparison table, emphasizing how it changes **the n^2 term to n** under the course's kernel assumptions.